

Student

My queries are turning into a mess of nested subqueries three levels deep. I cannot read my own SQL a week after writing it. Is there a better way to structure complex queries?

ChatGPT • AI Assistant

CONCEPT CARD

Yes - this is exactly the problem **CTEs (Common Table Expressions)** solve. A CTE lets you name an intermediate result and build your final query as a sequence of clearly labeled steps instead of a deeply nested block.

Basic syntax:

sql - Code Example

```
WITH cte_name AS (  
    SELECT ...  
)  
SELECT *  
FROM   cte_name  
WHERE  condition;
```

You define the CTE once with **WITH cte_name AS (...)**, then reference **cte_name** in the main query exactly like a table.

CONCEPT CARD**What a CTE actually is at runtime:**

In most databases (PostgreSQL 12+, SQL Server, Snowflake), a CTE is **NOT** a materialised temporary table by default. It is an **inline view** - the database optimizer substitutes the CTE definition directly into the main query at planning time, exactly like a regular view.

This means:

- The optimizer can push predicates down into the CTE.
- The optimizer can choose indexes on the underlying base tables as if the CTE were not there.

- A CTE referenced multiple times may be **recalculated each time** it is referenced (more on this later).

The value of a CTE is readability and logical decomposition - not performance. A CTE chain and an equivalent nested subquery often produce the identical execution plan. The difference is entirely in how easy the SQL is to read, debug, and maintain.

🗨️ ANALOGY

A deeply nested subquery is like a sentence with five clauses nested inside each other with no punctuation - technically parseable, but exhausting to read. A CTE chain is like breaking that sentence into short, labeled steps: 'First, do this.

Then, using that result, do this. Finally, combine them.' Same logic, same result - completely different cognitive load.

🌐 REAL WORLD APPLICATION

Every analytics engineer and data engineer writing SQL for dashboards, dbt models, or ad-hoc reporting uses CTE chains as the default structure for anything beyond a single JOIN. dbt models in particular are built almost entirely from chained CTEs - it is the standard style guide pattern across the industry.

Student

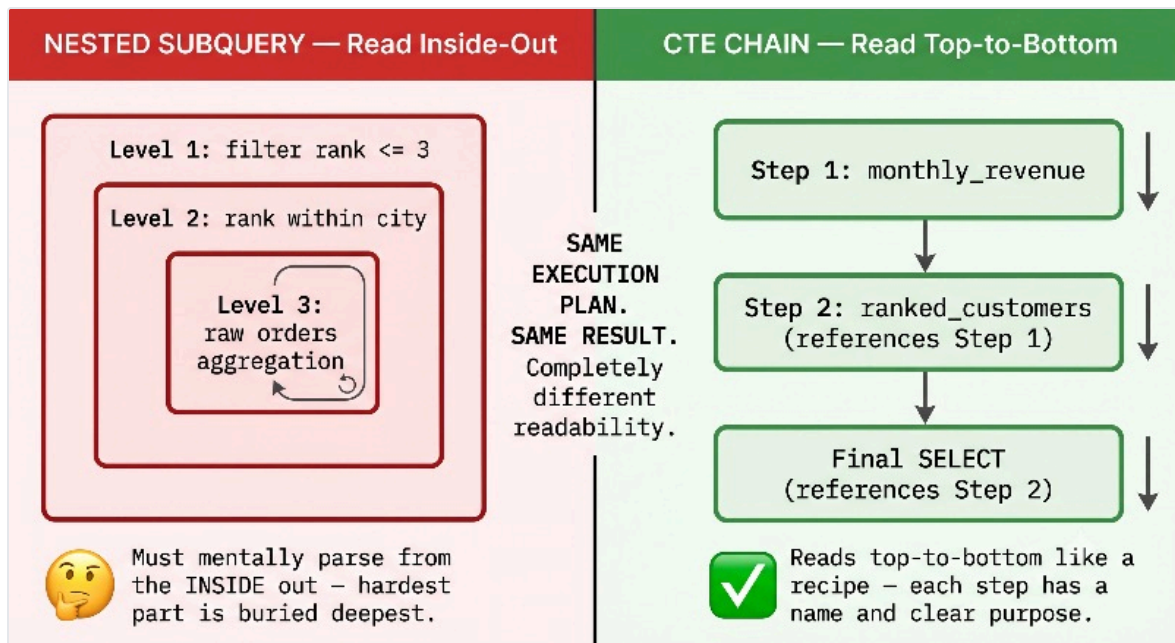
What schema are we using?

ChatGPT • AI Assistant

CONCEPT CARD

Same e-commerce schema used throughout this series:

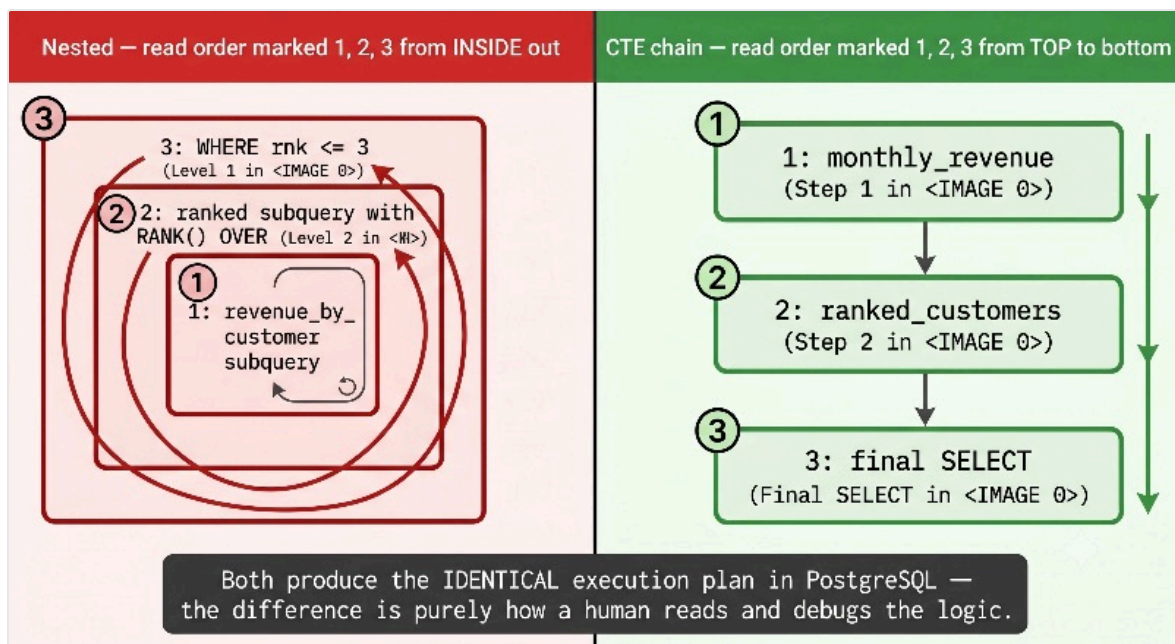
- `customers(customer_id, name, city, signup_date)`
- `orders(order_id, customer_id, order_date, total_amount, status)`
- `products(product_id, name, category, price)`
- `order_items(order_id, product_id, quantity, unit_price)`



Nested Subquery vs CTE Chain — Structural Comparison

FROM UNREADABLE NESTED SUBQUERY TO A CLEAN CTE CHAIN

ChatGPT • AI Assistant



Top-3-Per-City Query — Nested Reading Order vs CTE Reading Order

Show me the worst-case scenario - a query that is genuinely hard to read as nested subqueries, then the CTE version.

ChatGPT • AI Assistant

CONCEPT CARD

Here is a realistic example: **find the top 3 customers by revenue, per city.**

The unreadable nested version:

sql – Code Example

```
SELECT *
FROM (
    SELECT city, customer_id, customer_name, monthly_total,
           RANK() OVER (
               PARTITION BY city
               ORDER BY monthly_total DESC
           ) AS rnk
    FROM (
        SELECT c.city,
               c.customer_id,
               c.name AS customer_name,
               SUM(o.total_amount) AS monthly_total
        FROM customers c
        JOIN orders o ON c.customer_id = o.customer_id
        WHERE o.status ≠ 'cancelled'
              AND o.order_date ≥ DATE_TRUNC('month', CURRENT_DATE)
        GROUP BY c.city, c.customer_id, c.name
      ) AS revenue_by_customer
    ) AS ranked
WHERE rnk ≤ 3
ORDER BY city, rnk;
```

To understand this query you must read **from the innermost parentheses outward** - find the deepest SELECT first, mentally compute what it returns, then work your way up through two more layers. Even an experienced SQL developer needs a minute to trace this.

EXAMPLE CARD

The same logic rewritten as a CTE chain - step by step:

sql – Code Example

```

-- Step 1: compute revenue per customer per city, this month
WITH monthly_revenue AS (
    SELECT c.city,
           c.customer_id,
           c.name           AS customer_name,
           SUM(o.total_amount) AS monthly_total
    FROM   customers c
    JOIN   orders o ON c.customer_id = o.customer_id
    WHERE  o.status ≠ 'cancelled'
           AND o.order_date ≥ DATE_TRUNC('month', CURRENT_DATE)
    GROUP BY c.city, c.customer_id, c.name
),

-- Step 2: rank customers within each city by revenue
ranked_customers AS (
    SELECT city,
           customer_id,
           customer_name,
           monthly_total,
           RANK() OVER (
               PARTITION BY city
               ORDER BY monthly_total DESC
           ) AS rnk
    FROM   monthly_revenue
)

-- Final step: keep only the top 3 per city
SELECT city, customer_id, customer_name, monthly_total, rnk
FROM   ranked_customers
WHERE  rnk ≤ 3
ORDER BY city, rnk;

```

Read this top to bottom: 'First compute **monthly_revenue** . Then, using that, compute **ranked_customers** .

Finally, filter to the top 3.' Each step has a descriptive name. You can run **SELECT * FROM monthly_revenue** style debugging by temporarily changing the final SELECT to query an intermediate CTE name - something impossible with nested nested subqueries without copy-pasting the inner block out.

 WARNING

These two queries produce the **identical execution plan** in PostgreSQL. The CTE version is not faster - it is not slower either. The entire benefit is human readability

and the ability to debug each step independently.

INTERVIEW CARD

Frequency: Very High

Question: What is a CTE and how does it differ from a subquery in terms of performance?

Expected Answer: A CTE (Common Table Expression) is a named temporary result set defined with `WITH cte_name AS (...)` that can be referenced in the main query like a table. In most modern databases, a CTE is inlined by the optimizer exactly like a subquery - there is no inherent performance difference between a CTE chain and an equivalent nested subquery; both typically produce the same execution plan.

The value of a CTE is entirely readability: it lets you decompose a complex query into named, sequential, top-to-bottom steps instead of deeply nested parentheses that must be read inside-out.

Follow-up: Can you debug a CTE step independently? (Answer: Yes - temporarily change the final `SELECT` to `SELECT * FROM monthly_revenue` to inspect that one step's output, then change it back.

This kind of isolated debugging is far harder with deeply nested subqueries since the inner block is embedded inside outer parentheses.)

PRACTICE QUIZ

1. What keyword introduces a CTE?
2. Does a CTE physically store its result on disk by default?
3. What is the main benefit of using a CTE over an equivalent nested subquery?
4. In the example above, which CTE does `ranked_customers` depend on?

▼ **Reveal Correct Answer**

Answer Checklist: Check calculations, verify logic, check edge cases.

ANSWER KEY

1. `WITH` - followed by the CTE name and `AS (...)`.
2. No - by default a CTE is an inline view, substituted into the query at planning time, not a materialised temporary table.
3. Readability and logical decomposition - breaking a complex query into named, sequential steps that read top to bottom instead of nested inside-out.

4. **monthly_revenue** - it selects FROM monthly_revenue to compute the RANK() window function.

🔍 CHECKPOINT QUESTION

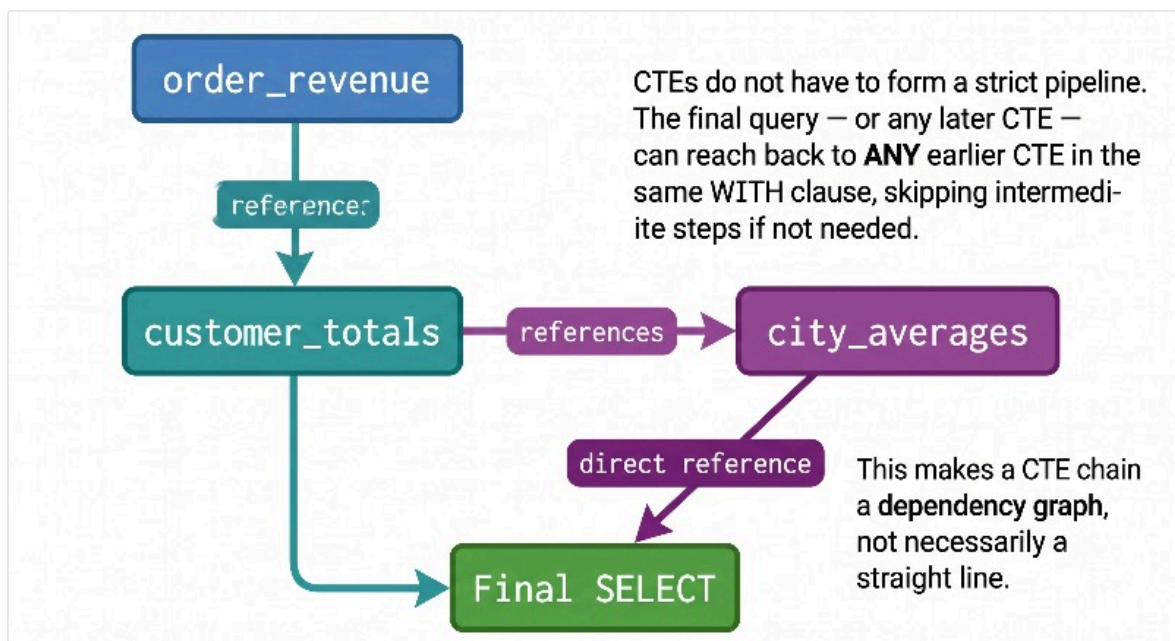
Rewrite this nested subquery as a two-step CTE chain: find all products whose total quantity sold (across order_items) is above the average quantity sold across all products.

sql – Code Example

```
SELECT * FROM (
    SELECT product_id, SUM(quantity) AS total_qty
    FROM order_items GROUP BY product_id
) t
WHERE total_qty > (
    SELECT AVG(total_qty) FROM (
        SELECT product_id, SUM(quantity) AS total_qty
        FROM order_items GROUP BY product_id
    ) t2
);
```

CHAINING MULTIPLE CTES AND REUSING A CTE TWICE

ChatGPT • AI Assistant



CTE Dependency Graph — Non-Linear References Across a Chain

Can I define more than two CTEs? And can a CTE later in the chain reference any CTE before it, not just the immediately previous one?

ChatGPT • AI Assistant

CONCEPT CARD

Yes - you can define **as many CTEs as you need**, separated by commas, all under a single **WITH** keyword. Each CTE can reference **any CTE defined earlier in the same WITH clause**, not just the immediately preceding one.

sql - Code Example

```
WITH cte_1 AS (  
    SELECT ...  
),  
cte_2 AS (  
    SELECT ... FROM cte_1 ...      -- can reference cte_1  
),  
cte_3 AS (  
    SELECT ... FROM cte_1 ...      -- can ALSO reference cte_1  
    directly  
    JOIN cte_2 ON ...             -- and reference cte_2 in the same  
    step  
)  
SELECT * FROM cte_3;
```

EXAMPLE CARD

Example 1 - A three-step chain where the third CTE uses two earlier CTEs:

sql - Code Example

```
WITH order_revenue AS (  
    -- Step 1: revenue per order  
    SELECT o.order_id,  
           o.customer_id,  
           o.order_date,  
           SUM(oi.quantity * oi.unit_price) AS order_total  
    FROM   orders o
```

```
JOIN    order_items oi ON o.order_id = oi.order_id
WHERE   o.status ≠ 'cancelled'
GROUP BY o.order_id, o.customer_id, o.order_date
),

customer_totals AS (
    -- Step 2: aggregate order_revenue per customer
    SELECT customer_id,
           SUM(order_total) AS lifetime_spend,
           COUNT(*)        AS order_count
    FROM   order_revenue    -- references Step 1
    GROUP BY customer_id
),

city_averages AS (
    -- Step 3: average lifetime spend per city,
    -- using BOTH customer_totals (Step 2) and the customers table
    directly
    SELECT c.city,
           AVG(ct.lifetime_spend) AS avg_city_spend
    FROM   customer_totals ct    -- references Step 2
    JOIN   customers c ON ct.customer_id = c.customer_id
    GROUP BY c.city
)

-- Final query: combine Step 2 and Step 3 to find above-average
customers
SELECT c.name,
       c.city,
       ct.lifetime_spend,
       ca.avg_city_spend
FROM   customer_totals ct    -- references Step 2 directly
JOIN   customers c  ON ct.customer_id = c.customer_id
JOIN   city_averages ca ON c.city = ca.city    -- references Step 3
WHERE  ct.lifetime_spend > ca.avg_city_spend
ORDER BY ct.lifetime_spend DESC;
```

Notice the final SELECT references **customer_totals** (Step 2) **directly**, skipping past **city_averages** (Step 3) - this is allowed. You are not restricted to a strict linear chain; any later step or the final query can reach back to any earlier CTE.

What if I need the same calculation twice in my final query - like comparing a value to itself in two different contexts?

ChatGPT • AI Assistant

CONCEPT CARD

Example 2 - Defining a CTE once and referencing it twice avoids repeating a complex calculation:

sql - Code Example

```
WITH product_revenue AS (  
    -- Complex calculation defined ONCE  
    SELECT p.product_id,  
           p.name      AS product_name,  
           p.category,  
           SUM(oi.quantity * oi.unit_price) AS total_revenue  
    FROM   products p  
    JOIN   order_items oi ON p.product_id = oi.product_id  
    JOIN   orders o      ON oi.order_id   = o.order_id  
    WHERE  o.status = 'delivered'  
    GROUP BY p.product_id, p.name, p.category  
)  
  
-- Reference product_revenue TWICE in the main query:  
SELECT pr.product_name,  
       pr.category,  
       pr.total_revenue,  
       -- First reference: compare to category average  
       (SELECT AVG(total_revenue)  
        FROM   product_revenue pr2  
        WHERE  pr2.category = pr.category) AS  
category_avg_revenue,  
       -- Second reference: compare to overall average  
       (SELECT AVG(total_revenue) FROM product_revenue) AS  
overall_avg_revenue  
FROM   product_revenue pr  
ORDER BY pr.total_revenue DESC;
```

Without the CTE, you would need to repeat the entire JOIN + GROUP BY logic three separate times - once for the main row, once for the category average subquery, and

once for the overall average subquery. Define it once in the CTE, reference it as many times as needed.

⚠ WARNING

Referencing a non-materialised CTE multiple times means the database **may re-execute the underlying query each time it is referenced** - in the example above, `product_revenue` could be computed up to three times (once for `pr`, once for `pr2` in the correlated subquery, once for the plain AVG). This is a readability win, but check the EXPLAIN plan if the underlying CTE query is expensive and referenced many times - covered in detail in the next subsection.

🔗 INTERVIEW CARD

Frequency: High

Question: Can a CTE reference another CTE defined earlier in the same WITH clause? Can the final query skip a CTE and reference an earlier one directly?

Expected Answer: Yes to both. CTEs are defined in sequence within a single WITH clause, separated by commas, and each CTE can reference any CTE defined before it - not just the immediately preceding one.

The final SELECT (or the last CTE in the chain) can reference any CTE in the chain, including ones several steps back, skipping intermediate CTEs if needed. This flexibility lets you structure a chain not as a strict linear pipeline but as a small dependency graph where later steps pull from whichever earlier results they need.

Follow-up: What is the risk of referencing the same CTE multiple times in the main query? (Answer: Without materialisation, the database may recompute the CTE's underlying query once per reference, multiplying the cost of an expensive CTE.

Check EXPLAIN to see whether the plan reuses one computed result or repeats the subquery - and consider the MATERIALIZED hint or a temp table if it repeats expensively.)

❓ PRACTICE QUIZ

1. Can `cte_3` in a chain reference `cte_1` even if `cte_2` is defined in between?
2. In the `customer_totals / cityaverages` example, which CTE does the final SELECT reference directly without going through `city_averages` first?
3. What is the benefit of defining a complex calculation once in a CTE and referencing it multiple times in the main query?
4. What is the performance risk of referencing the same CTE multiple times?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

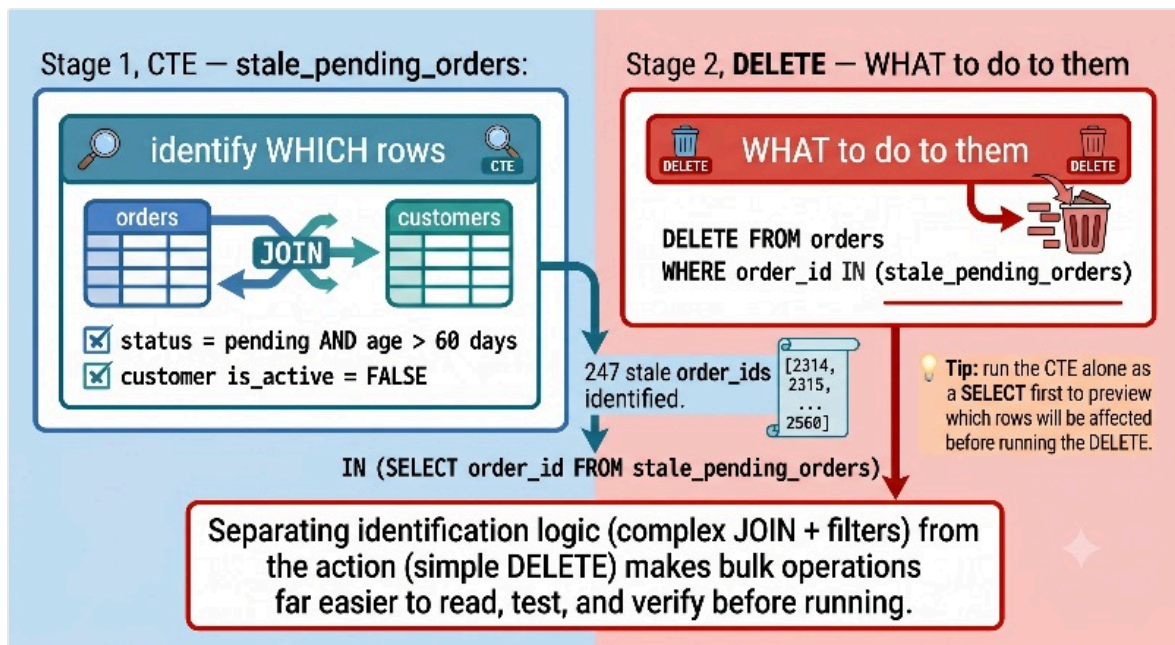
1. Yes - any CTE can reference any CTE defined earlier in the same WITH clause, regardless of position.
2. `customer_totals` - the final SELECT joins to it directly (`FROM customer_totals ct JOIN customers c ...`) while also separately joining to `city_averages` .
3. You avoid repeating the same JOIN and GROUP BY logic multiple times in the query text - write it once, reference it as many times as needed, improving readability and reducing the chance of inconsistency between repeated copies.
4. The database may recompute the CTE's underlying query once per reference (since it is not materialised by default), multiplying the execution cost of an expensive CTE by the number of times it is referenced.

❓ CHECKPOINT QUESTION

Write a three-CTE chain where: Step 1 (`daily_orders`) computes order count and revenue per day; Step 2 (`weekly_avg`) computes the average daily revenue per week from Step 1; and the final query joins Step 1 directly to Step 2 to find days where revenue was more than double that week's average. Show how the final query references Step 1 directly even though Step 2 depends on Step 1.

THE WITH CLAUSE IN INSERT, UPDATE, AND DELETE STATEMENTS

ChatGPT • AI Assistant



CTE-Driven DELETE — Separating 'Which Rows' Logic from 'What To Do' Logic

Student

I always think of CTEs as a SELECT-only feature. Can they be used with INSERT, UPDATE, or DELETE too?

ChatGPT • AI Assistant

CONCEPT CARD

Yes - the **WITH** clause works with **INSERT**, **UPDATE**, and **DELETE** exactly the same way it works with **SELECT**. You define the CTE once, then reference it in the data-modifying statement that follows.

This is especially powerful for **DELETE** and **UPDATE** operations where you need complex logic to first *identify* which rows are affected, before acting on them.

EXAMPLE CARD**Example 1 - CTE that identifies stale records, feeding directly into a DELETE:**

sql – Code Example

```
WITH stale_pending_orders AS (
  -- Identify orders that have been 'pending' for over 60 days
  -- and the customer account is no longer active
  SELECT o.order_id
```

```
FROM    orders o
JOIN    customers c ON o.customer_id = c.customer_id
WHERE   o.status = 'pending'
        AND o.order_date < CURRENT_DATE - INTERVAL '60 days'
        AND c.is_active = FALSE
)
DELETE FROM orders
WHERE   order_id IN (SELECT order_id FROM stale_pending_orders);
```

The CTE does the complex identification work - joining to customers, checking two conditions - and the DELETE statement stays simple: delete by order_id IN (the CTE result).

Example 2 - CTE feeding an UPDATE: apply a tier upgrade to qualifying customers

sql – Code Example

```
WITH qualifying_customers AS (
    SELECT customer_id
    FROM    orders
    WHERE   status = 'delivered'
    GROUP BY customer_id
    HAVING SUM(total_amount) ≥ 50000
)
UPDATE customers
SET     tier = 'Gold'
WHERE   customer_id IN (SELECT customer_id FROM qualifying_customers)
        AND tier ≠ 'Gold';  -- avoid no-op updates on already-Gold
customers
```

Example 3 - CTE feeding an INSERT: archive old delivered orders

sql – Code Example

```
WITH orders_to_archive AS (
    SELECT order_id, customer_id, order_date, total_amount, status
    FROM    orders
    WHERE   status = 'delivered'
        AND order_date < CURRENT_DATE - INTERVAL '365 days'
)
INSERT INTO orders_archive (order_id, customer_id, order_date,
total_amount, status)
SELECT order_id, customer_id, order_date, total_amount, status
FROM    orders_to_archive;
```

```
-- Companion DELETE using the same CTE pattern
WITH orders_to_archive AS (
    SELECT order_id
    FROM    orders
    WHERE   status = 'delivered'
           AND order_date < CURRENT_DATE - INTERVAL '365 days'
)
DELETE FROM orders
WHERE  order_id IN (SELECT order_id FROM orders_to_archive);
```

Example 4 - PostgreSQL specific: RETURNING + a CTE to chain a DELETE into an INSERT in one statement

sql – Code Example

```
WITH deleted_rows AS (
    DELETE FROM orders
    WHERE   status = 'cancelled'
           AND order_date < CURRENT_DATE - INTERVAL '180 days'
    RETURNING order_id, customer_id, order_date, total_amount, status
)
INSERT INTO orders_archive (order_id, customer_id, order_date,
total_amount, status)
SELECT order_id, customer_id, order_date, total_amount, status
FROM    deleted_rows;
-- One statement: deletes cancelled orders AND archives them
atomically.
-- This is PostgreSQL's 'data-modifying CTE' feature – DELETE itself
can be the CTE body.
```

⚠ WARNING

Using a DELETE or UPDATE **inside** the CTE body (as in Example 4) is a PostgreSQL-specific feature called a **data-modifying CTE**. Not all databases support this - Snowflake and many others only support read-only SELECT inside the WITH clause feeding a separate INSERT/UPDATE/DELETE statement (as in Examples 1-3), which is the more portable pattern.

📖 INTERVIEW CARD

Frequency: Medium-High

Question: Can you use a CTE with DELETE or UPDATE? Give an example of why this is useful.

Expected Answer: Yes - the WITH clause is not restricted to SELECT; it works identically with INSERT, UPDATE, and DELETE. The typical pattern is to use the CTE to identify the set of rows that need to be affected using whatever complex JOIN, aggregation, or filtering logic is needed, then reference that CTE's result in the WHERE clause of the data-modifying statement using IN (SELECT ...

FROM cte). This separates the complex 'which rows' logic from the simple 'what to do to them' logic, making bulk update/delete operations far more readable than embedding the same logic inline in the WHERE clause.

Follow-up: What is a 'data-modifying CTE' in PostgreSQL? (Answer: A PostgreSQL-specific feature where the CTE body itself contains a DELETE, UPDATE, or INSERT statement with a RETURNING clause, and a subsequent CTE or the main query consumes those returned rows - for example, deleting rows and inserting the deleted data into an archive table in one atomic statement.

This is not universally portable across all databases.)

🔗 PRACTICE QUIZ

1. Does the WITH clause work only with SELECT statements?
2. In Example 1, what two conditions does the CTE check before identifying an order as stale?
3. Why might you add `AND tier ≠ 'Gold'` to the UPDATE in Example 2?
4. What is the key difference between the data-modifying CTE pattern in Example 4 and the patterns in Examples 1-3?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. No - WITH works identically with SELECT, INSERT, UPDATE, and DELETE.
2. The order status must be 'pending' for over 60 days, AND the associated customer's account must be inactive (is_active = FALSE).
3. To avoid a no-op UPDATE that rewrites a row's tier to the same value it already has - this prevents unnecessary write I/O and avoids triggering update-related triggers or audit logs for rows that did not actually change.
4. In Examples 1-3, the CTE body is read-only (a SELECT) and the actual data modification happens in a separate statement that references the CTE.

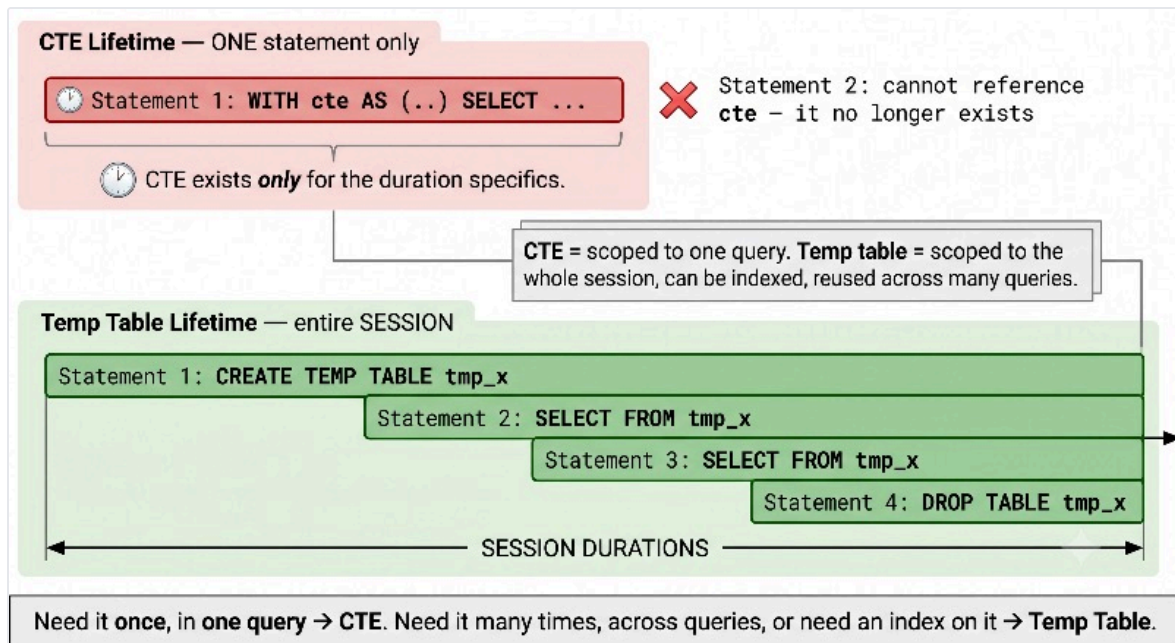
In Example 4, the CTE body itself IS the data-modifying statement (a DELETE with RETURNING), and its output rows are consumed by a following statement - a PostgreSQL-specific 'data-modifying CTE' feature, less portable across databases.

🔍 CHECKPOINT QUESTION

Write a CTE-driven UPDATE that increases stock_{qty} by 10% (rounded down to the nearest integer) for all products in the 'Electronics' category whose total quantity sold (from orderitems, delivered orders only) is below 50 units - i.e., restock slow-moving electronics. Use a CTE to first identify the qualifying product_ids, then UPDATE products using that CTE.

WHEN NOT TO USE A CTE — MATERIALIZED TEMP TABLES AND THE MATERIALIZED HINT

ChatGPT • AI Assistant



CTE Lifetime vs Temp Table Lifetime — Single Statement vs Session Scope

Student

You mentioned a CTE might be recalculated every time it is referenced. When should I NOT use a CTE, and what should I use instead?

CONCEPT CARD

There are two specific situations where a plain CTE is the wrong tool:

Situation 1 - You need the intermediate result physically materialised for performance

If an intermediate calculation is expensive (a large aggregation, a heavy multi-table JOIN) and you need to use it multiple times across several queries - not just once within a single statement - a CTE will not help, because a CTE only lives for the duration of one SQL statement. Use a **temporary table** instead:

sql – Code Example

```
-- WRONG TOOL: a CTE cannot persist across multiple separate
statements
WITH expensive_summary AS (
    SELECT customer_id, SUM(total_amount) AS lifetime_spend
    FROM    orders
    GROUP BY customer_id
)
SELECT * FROM expensive_summary WHERE lifetime_spend > 10000;
-- If you run a SECOND query needing expensive_summary, it must be
-- recomputed from scratch – the CTE does not persist between
statements.

-- RIGHT TOOL: a temporary table persists for the whole session
CREATE TEMP TABLE tmp_expensive_summary AS
SELECT customer_id, SUM(total_amount) AS lifetime_spend
FROM    orders
GROUP BY customer_id;

CREATE INDEX idx_tmp_summary_spend ON
tmp_expensive_summary(lifetime_spend);

-- Now reuse it across multiple unrelated queries in the same session:
SELECT * FROM tmp_expensive_summary WHERE lifetime_spend > 10000;
SELECT AVG(lifetime_spend) FROM tmp_expensive_summary;
SELECT customer_id FROM tmp_expensive_summary ORDER BY lifetime_spend
DESC LIMIT 10;
-- Computed ONCE, reused as many times as needed, can even be indexed.
DROP TABLE tmp_expensive_summary; -- clean up when done
```

Situation 2 - The CTE is referenced many times within one complex query and gets recalculated each time

sql – Code Example

```
-- A CTE referenced 3 times in the SAME query may execute its
-- underlying SELECT up to 3 separate times (database-dependent):
WITH expensive_calc AS (
    SELECT product_id, SUM(quantity * unit_price) AS revenue
    FROM   order_items GROUP BY product_id      -- expensive: scans
    millions of rows
)
SELECT e1.product_id,
       e1.revenue,
       (SELECT AVG(revenue) FROM expensive_calc)      AS
overall_avg, -- reference 2
       (SELECT MAX(revenue) FROM expensive_calc)      AS overall_max
-- reference 3
FROM   expensive_calc e1; -- reference 1
-- Without materialisation: the GROUP BY over order_items may run 3
times!
```

Student

How do I force PostgreSQL to compute the CTE only once?

ChatGPT • AI Assistant

CONCEPT CARD

Use the **MATERIALIZED** hint (PostgreSQL 12+):

sql – Code Example

```
WITH expensive_calc AS MATERIALIZED (
    SELECT product_id, SUM(quantity * unit_price) AS revenue
    FROM   order_items GROUP BY product_id
)
SELECT e1.product_id,
       e1.revenue,
       (SELECT AVG(revenue) FROM expensive_calc) AS overall_avg,
       (SELECT MAX(revenue) FROM expensive_calc) AS overall_max
FROM   expensive_calc e1;
-- MATERIALIZED forces PostgreSQL to compute expensive_calc EXACTLY
```

```
ONCE,  
-- store the result in memory/temp space for the duration of this  
query,  
-- and reuse it for every reference. The opposite of MATERIALIZED is  
-- NOT MATERIALIZED, which forces inlining (the pre-12 default  
behaviour).
```

CONCEPT CARD

PostgreSQL's default behaviour changed in version 12:

code – Code Example

```
PostgreSQL < 12: CTEs were ALWAYS materialised (computed once,  
always)  
→ safe but sometimes slower (optimizer couldn't push  
predicates through the CTE boundary)  
  
PostgreSQL ≥ 12: CTEs are inlined by default UNLESS:  
- the CTE is referenced more than once, OR  
- the CTE contains a data-modifying statement  
(INSERT/UPDATE/DELETE), OR  
- you explicitly write MATERIALIZED
```

In practice: PostgreSQL 12+ already auto-materialises a CTE that is referenced multiple times in many cases - but this is a heuristic, not a guarantee. Always check **EXPLAIN ANALYZE** if performance matters, and use the explicit **MATERIALIZED** keyword when you need certainty rather than relying on the optimizer's default choice.

⚠ WARNING

MATERIALIZED is a PostgreSQL-specific syntax extension. Other databases (Snowflake, SQL Server, MySQL 8+) have their own optimizer behaviour for CTE reuse and do not support this exact keyword - check your platform's documentation.

The underlying principle - 'does this engine compute the CTE once or every time it is referenced?' - is universal even when the syntax to control it differs.

🔗 INTERVIEW CARD

Frequency: High

Question: When would you choose a temporary table over a CTE? What is the **MATERIALIZED** hint for?

Expected Answer: Choose a temporary table when the intermediate result needs to persist across multiple separate SQL statements in the same session - a CTE only

exists for the duration of one statement and cannot be reused across statements. Also choose a temp table when you need to index the intermediate result for fast repeated lookups, since a plain CTE cannot be indexed.

The **MATERIALIZED** hint in PostgreSQL forces a CTE referenced multiple times within a single query to be computed exactly once and reused, rather than potentially being recalculated on each reference - useful when the CTE's underlying query is expensive and the optimizer's default inlining behaviour would otherwise repeat that expensive computation multiple times.

Follow-up: Did PostgreSQL always inline CTEs the way it does today? (Answer: No - before PostgreSQL 12, every CTE was always materialised by default with no option to inline.

PostgreSQL 12 flipped the default to inline CTEs unless they are referenced multiple times, contain a data-modifying statement, or are explicitly marked **MATERIALIZED** - generally improving performance by allowing predicate pushdown through CTE boundaries.)

🔍 PRACTICE QUIZ

1. Can a CTE be reused across two separate, unrelated SQL statements run back-to-back in the same session?
2. What PostgreSQL keyword forces a CTE to be computed exactly once even if referenced multiple times?
3. Can you create an index directly on a CTE?
4. Before PostgreSQL 12, were CTEs inlined or always materialised by default?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

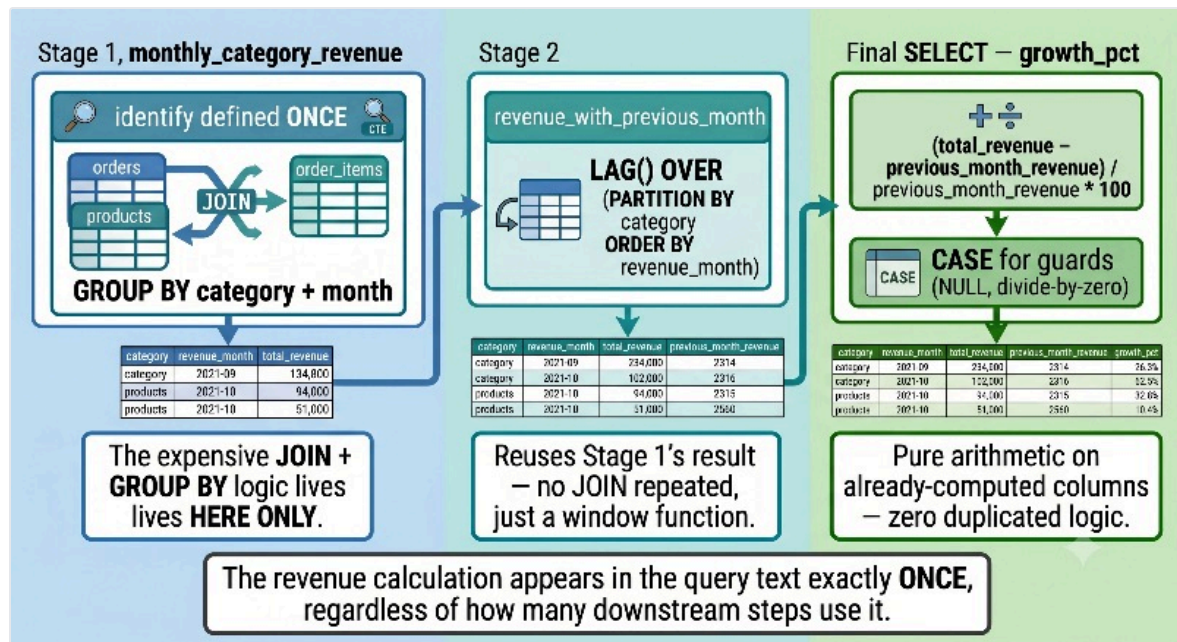
1. No - a CTE's scope is limited to the single SQL statement it is defined in. For reuse across multiple statements, use a temporary table instead.
2. **MATERIALIZED** - written as **WITH cte_name AS MATERIALIZED (SELECT ...)**.
3. No - a CTE is not a real table object, so you cannot **CREATE INDEX** on it. If you need an index on an intermediate result, materialise it into a temp table first.
4. Always materialised by default - PostgreSQL versions before 12 had no inlining behaviour; every CTE was computed once and stored for the query's duration.

🔍 CHECKPOINT QUESTION

You have a CTE `category_stats` that performs an expensive 3-table JOIN and GROUP BY, and it is referenced 4 times in your final SELECT (once for the main row, three times in correlated scalar subqueries for different comparisons). Explain the two ways you could ensure this expensive calculation runs only once, and the trade-offs between them.

PRACTICE PROBLEMS — REWRITING NESTED SUBQUERIES AND MONTH-OVER-MONTH GROWTH

ChatGPT • AI Assistant



Month-over-Month Revenue Growth — CTE Chain with Single-Source Revenue Logic

Student

Let's work through the two practice problems.

ChatGPT • AI Assistant

CONCEPT CARD

Practice Problem 1 - Rewrite an ugly nested subquery as a clean CTE chain:

Given query (find customers whose average order value this year is higher than the average order value of customers in the same city last year):

sql – Code Example

```
SELECT this_year.customer_id, this_year.name, this_year.avg_this_year
FROM (
    SELECT c.customer_id, c.name, c.city,
           AVG(o.total_amount) AS avg_this_year
    FROM customers c
    JOIN orders o ON c.customer_id = o.customer_id
    WHERE EXTRACT(YEAR FROM o.order_date) = EXTRACT(YEAR FROM
CURRENT_DATE)
    AND o.status ≠ 'cancelled'
    GROUP BY c.customer_id, c.name, c.city
) this_year
JOIN (
    SELECT city, AVG(total_amount) AS avg_last_year
    FROM customers c2
    JOIN orders o2 ON c2.customer_id = o2.customer_id
    WHERE EXTRACT(YEAR FROM o2.order_date) = EXTRACT(YEAR FROM
CURRENT_DATE) - 1
    AND o2.status ≠ 'cancelled'
    GROUP BY city
) last_year_by_city
ON this_year.city = last_year_by_city.city
WHERE this_year.avg_this_year > last_year_by_city.avg_last_year;
```

Solution - CTE chain:

sql – Code Example

```
WITH this_year_avg AS (
    -- Step 1: average order value per customer, this year
    SELECT c.customer_id,
           c.name,
           c.city,
           AVG(o.total_amount) AS avg_this_year
    FROM customers c
    JOIN orders o ON c.customer_id = o.customer_id
    WHERE EXTRACT(YEAR FROM o.order_date) = EXTRACT(YEAR FROM
CURRENT_DATE)
    AND o.status ≠ 'cancelled'
    GROUP BY c.customer_id, c.name, c.city
),
```

```
last_year_city_avg AS (  
    -- Step 2: average order value per CITY, last year  
    SELECT c.city,  
           AVG(o.total_amount) AS avg_last_year  
    FROM   customers c  
    JOIN   orders o ON c.customer_id = o.customer_id  
    WHERE  EXTRACT(YEAR FROM o.order_date) = EXTRACT(YEAR FROM  
CURRENT_DATE) - 1  
           AND o.status ≠ 'cancelled'  
    GROUP BY c.city  
)  
  
-- Final step: customers beating their city's last-year average  
SELECT ty.customer_id,  
       ty.name,  
       ty.city,  
       ty.avg_this_year,  
       ly.avg_last_year  
FROM   this_year_avg ty  
JOIN   last_year_city_avg ly ON ty.city = ly.city  
WHERE  ty.avg_this_year > ly.avg_last_year  
ORDER BY ty.avg_this_year DESC;
```

Each CTE has a name describing exactly what it computes - `this_year_avg` , `last_year_city_avg` - replacing the meaningless `this_year` / `last_year_by_city` subquery aliases from the original, and removing one level of visual nesting entirely.

Practice Problem 2 - Month-over-month revenue growth per product category, without repeating the revenue calculation:

Requirement: for each product category and month, show total revenue, the previous month's revenue for that same category, and the percentage growth - using the revenue calculation only once.

Solution:

sql – Code Example

```
WITH monthly_category_revenue AS (  
    -- Step 1: revenue per category per month – defined ONCE  
    SELECT p.category,  
           DATE_TRUNC('month', o.order_date) AS revenue_month,  
           SUM(oi.quantity * oi.unit_price) AS total_revenue  
    FROM   orders o
```

```

JOIN    order_items oi ON o.order_id    = oi.order_id
JOIN    products p    ON oi.product_id = p.product_id
WHERE   o.status = 'delivered'
GROUP BY p.category, DATE_TRUNC('month', o.order_date)
),

revenue_with_previous_month AS (
    -- Step 2: use a window function (LAG) to pull in the previous
    month's
    -- revenue for the SAME category – referencing Step 1, not
    repeating the JOIN
    SELECT category,
           revenue_month,
           total_revenue,
           LAG(total_revenue) OVER (
               PARTITION BY category
               ORDER BY revenue_month
           ) AS previous_month_revenue
    FROM   monthly_category_revenue
)

-- Final step: compute growth percentage
SELECT category,
       revenue_month,
       total_revenue,
       previous_month_revenue,
       CASE
           WHEN previous_month_revenue IS NULL THEN NULL -- no prior
           month to compare
           WHEN previous_month_revenue = 0      THEN NULL -- avoid
           division by zero
           ELSE ROUND(
               (total_revenue - previous_month_revenue)
               / previous_month_revenue * 100, 2
           )
       END AS growth_pct
FROM   revenue_with_previous_month
ORDER BY category, revenue_month;

```

The revenue calculation - the JOIN across `orders` , `order_items` , and `products` , plus the GROUP BY - is written **exactly once**, in `monthly_category_revenue` . The second CTE reuses that result purely to apply a window function.

The final SELECT does pure arithmetic on already-computed columns. No part of the underlying revenue logic is duplicated anywhere in the query.

SUMMARY CARD

SQL 1.11 Key Takeaways:

- **CTE syntax:** `WITH cte_name AS (SELECT ... ) SELECT ... FROM cte_name .`
- **At runtime:** a CTE is typically an inline view, not a materialised temp table - performance is usually identical to an equivalent nested subquery.
- **The value of a CTE is readability** - turning inside-out nested parentheses into a top-to-bottom sequence of named, debuggable steps.
- **Chain as many CTEs as needed**, separated by commas; any CTE (or the final query) can reference any earlier CTE, not just the one immediately before it.
- **Define a complex calculation once, reference it multiple times** in the main query to avoid duplicating logic.
- **WITH works with INSERT, UPDATE, DELETE too** - use a CTE to identify which rows are affected, then act on them in a simple statement referencing the CTE.
- **PostgreSQL data-modifying CTEs** let a DELETE/UPDATE/INSERT live inside the CTE body itself with RETURNING - a non-portable but powerful pattern for atomic delete-then-archive operations.
- **Use a temp table instead of a CTE** when you need the result to persist across multiple statements, or need to index it.
- **Use MATERIALIZED** (PostgreSQL 12+) to force a multiply-referenced CTE to compute exactly once within a single query.
- **Recursive CTEs are a separate, more advanced topic** - covered next in SQL 3.12 - they unlock hierarchical and iterative problems that plain CTEs cannot solve.

🔮 AI DEEP DIVE PROMPT

Copy this prompt into Claude or ChatGPT for deeper practice:

'Act as a senior SQL mentor. I just learned CTEs - WITH syntax, chaining multiple CTEs, referencing earlier CTEs from later ones, CTEs with INSERT/UPDATE/DELETE, and the MATERIALIZED hint.

Give me 8 progressively harder exercises. Start with rewriting a 2-level nested subquery as a 2-step CTE chain, build up to a 4-step CTE chain combining window functions and aggregation, and finish with a CTE-driven UPDATE that identifies and modifies rows based on a multi-table condition.

For each exercise: give me the messy version first, ask me to rewrite it as a CTE chain, then show the correct solution, explain the naming choices for each CTE, and give one interview question about CTE behaviour. Do not continue until I submit my answer.'

🔍 CHECKPOINT QUESTION

Before moving to SQL 3.12 (Recursive CTEs): without running it, identify what is wrong with this CTE chain and write the corrected version:

sql – Code Example

```
WITH step_one AS (  
    SELECT customer_id, SUM(total_amount) AS spend  
    FROM orders GROUP BY customer_id  
)  
,  
step_two AS (  
    SELECT customer_id, spend  
    FROM step_three -- references a CTE not yet defined  
    WHERE spend > 1000  
)  
,  
step_three AS (  
    SELECT * FROM step_one  
)  
SELECT * FROM step_two;
```